

Regularization

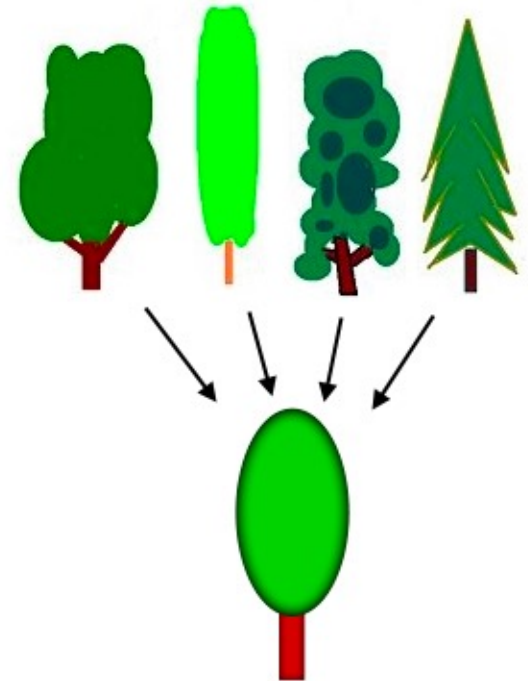
Gunhee Kim



서울대학교
SEOUL NATIONAL UNIVERSITY

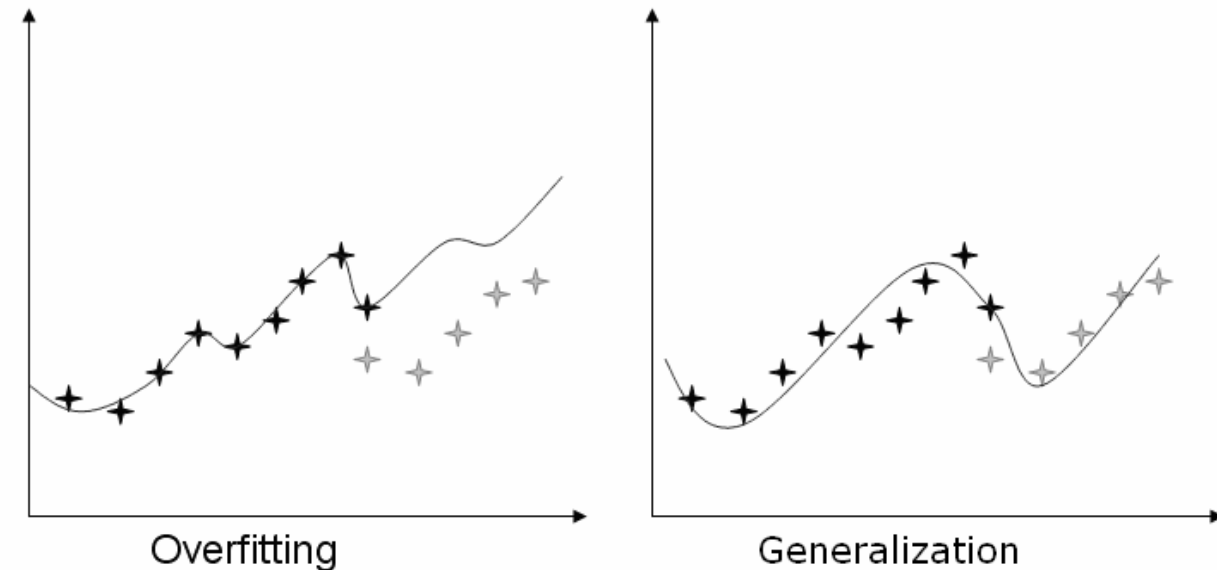
Generalization

- Objective of learning
 - Not to learn an exact representation of the training data itself
 - To build a statistical model of the process that generates the data
- Generalization
 - A form of abstraction where common properties of specific instances are formulated as general concepts or claims
 - It extracts the essence of a concept based on its analysis of similarities from many discrete objects
 - If a toddler had never seen a willow tree or pine tree before he still might classify it as a tree because it is green

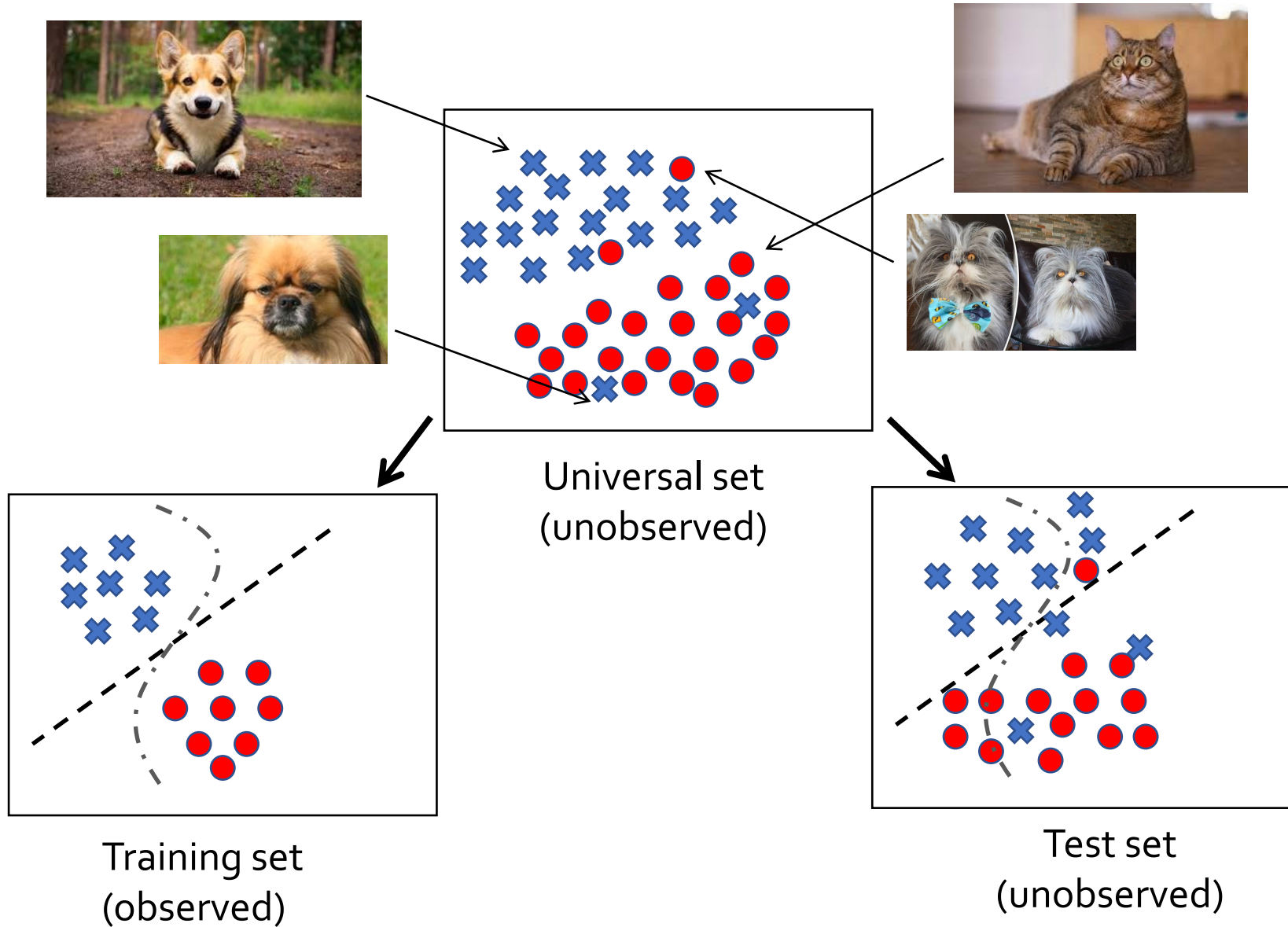


Generalization

- Generalization in ML
 - An ML model's ability to perform well on new unseen data rather than just the data that it was trained on
 - Learning algorithm maximizes accuracy on training examples
 - How to generalize from training to test?
- Strongly related to the concept of overfitting
 - Overfitting = poor generalization



Training Data vs Test Data



Generalization Error

- True distribution: $P(x, y)$
 - All possible cases – unknown to us
 - Training and test data are generated by $P(x, y)$
 - Assumption: i.i.d. (independent and identically distributed)
- Training: fit a hypothesis $h(x)$
 - Using training data $S = \{(x_i, y_i)\}_{i=1}^N$, sampled from $P(x, y)$
- Generalization error: $L_P(h) = \mathbb{E}_{P(x,y)}[L(y, h(x))]$
 - Prediction loss on all possible cases
 - Overfitting: Generalization error > Training error
 - Underfitting: Generalization error < Training error

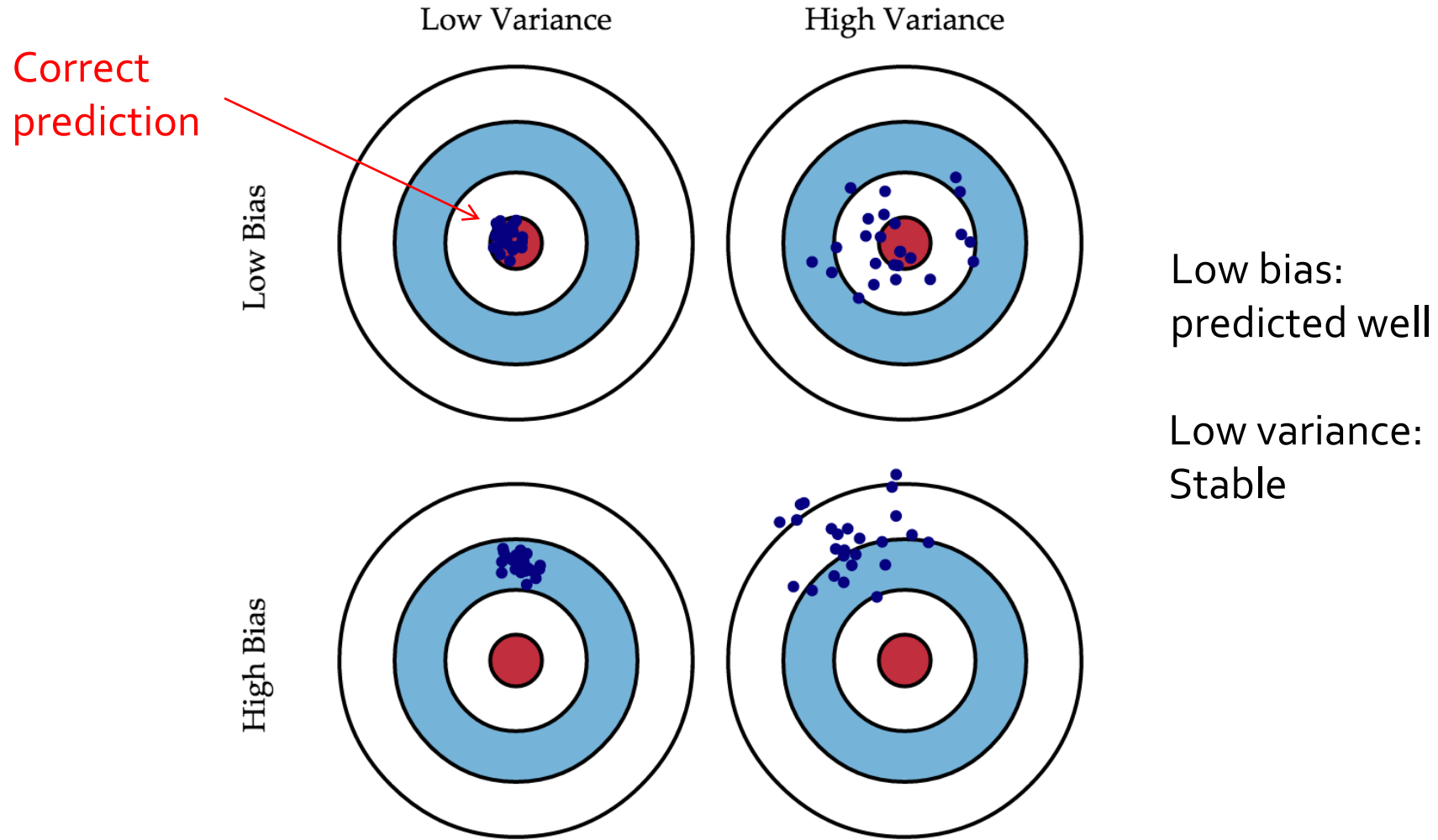
Bias/Variance Decomposition

- Test error $L_P(h) = \mathbb{E}_{P(x,y)}[L(y^*, h(x^*))]$
 - Given a new test sample (x^*, y^*) where $y^* = f(x^*) + \varepsilon$
 - Squared loss $L(a, b) = (a - b)^2$

$$\begin{aligned}\mathbb{E}[(y^* - h(x^*))^2] &= \mathbb{E}[(y^* - f(x^*))^2] && \text{(noise)} \\ &+ (f(x^*) - \mathbb{E}[h(x^*)])^2 && \text{(Bias)}^2 \\ &+ \mathbb{E}[(h(x^*) - \mathbb{E}[h(x^*)])^2] && \text{(Variance)}\end{aligned}$$

- Average prediction $H(x^*) = \mathbb{E}[h(x^*)]$
- Variance term: how much each model varies from one training set to another
- Bias: describes the average error of each data

Bias/Variance Decomposition



Overfitting vs Underfitting

- High variance implies **overfitting**
 - Model class unstable
 - Variance increases with model complexity
 - Variance reduces with more training data
- High bias implies **underfitting**
 - Even with no variance, model class has high error
 - Bias decreases with model complexity
 - Independent of training data size

Regularization

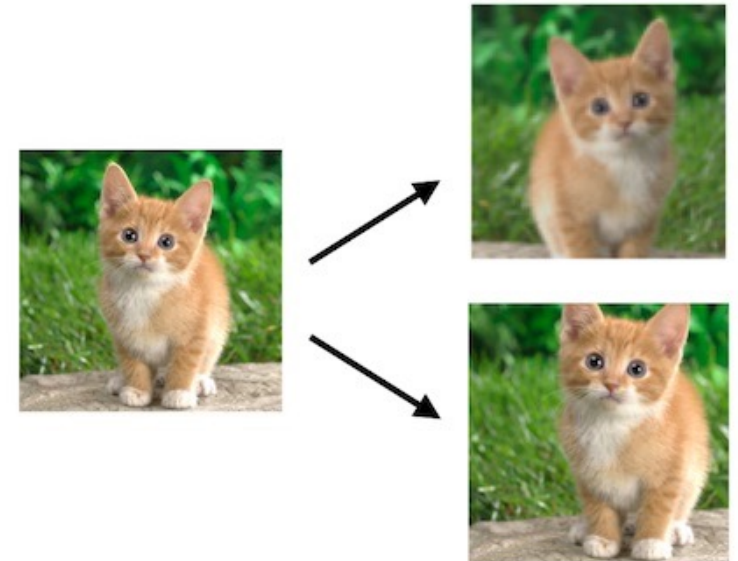
- Any modification we make to a ML algorithm that is intended to reduce its generalization error but not its training error
- Several strategy
 - Extra constraints on a machine learning algorithm
 - Extra terms in the objective (based on prior knowledge)
 - Ensemble methods (combine multiple hypotheses that explain the training data)

Ensemble Methods for Neural Networks

- Highly recommended!
 - Model averaging is an extremely powerful and reliable for reducing generalization error
- NNs reach a wide variety of solution points
 - Never obtain the same solution at each running
 - Random initialization, random selection of minibatches, different hyperparameter setting, ...

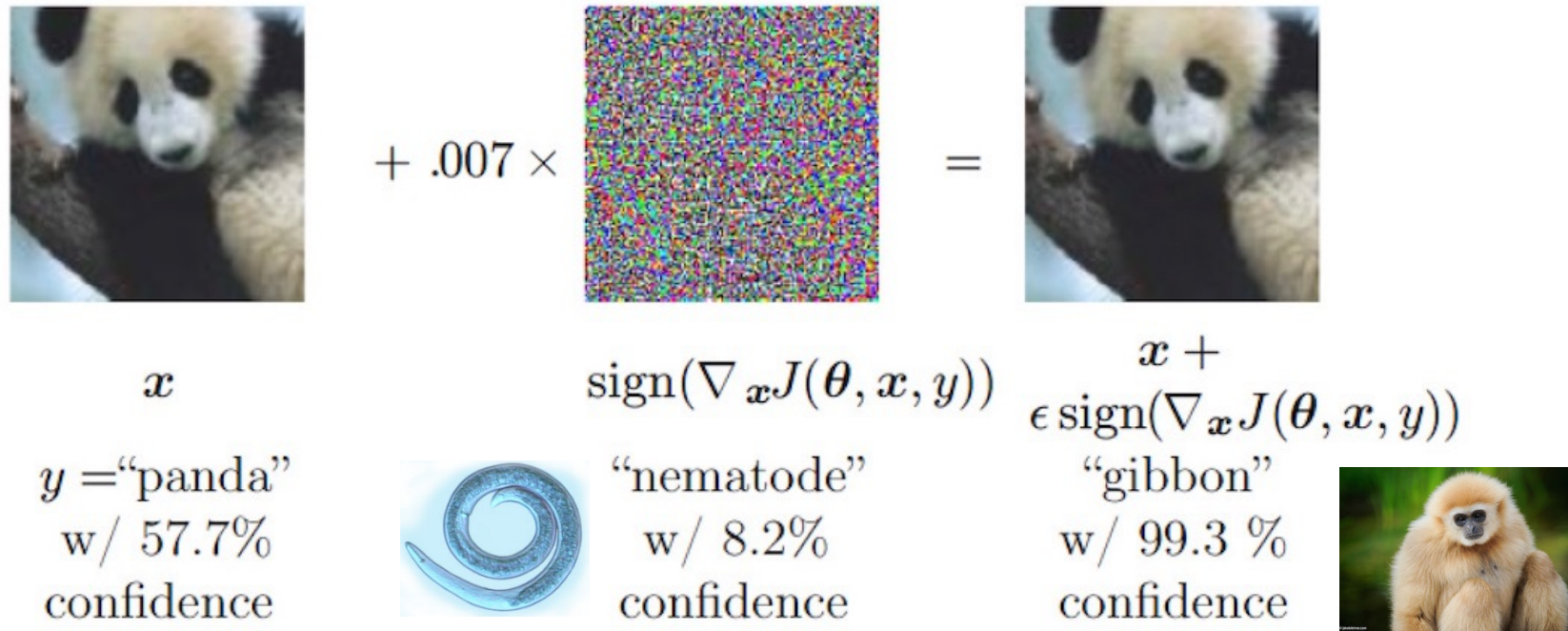
Data Augmentation

- The best way to make ML model generalize better: more training data
- Create fake data from training data!
 - For a training sample (x, y) , make some transformation (x', y)
- Very effective for image classification
 - Images are high-dimensional and have an enormous variety of factors of variation
 - (Partial) translating by a few pixels, rotating, scaling, and flipping



Noise Injection

- Neural networks are not robust against noise
 - e.g., Fast Gradient Sign Method (FGSM)

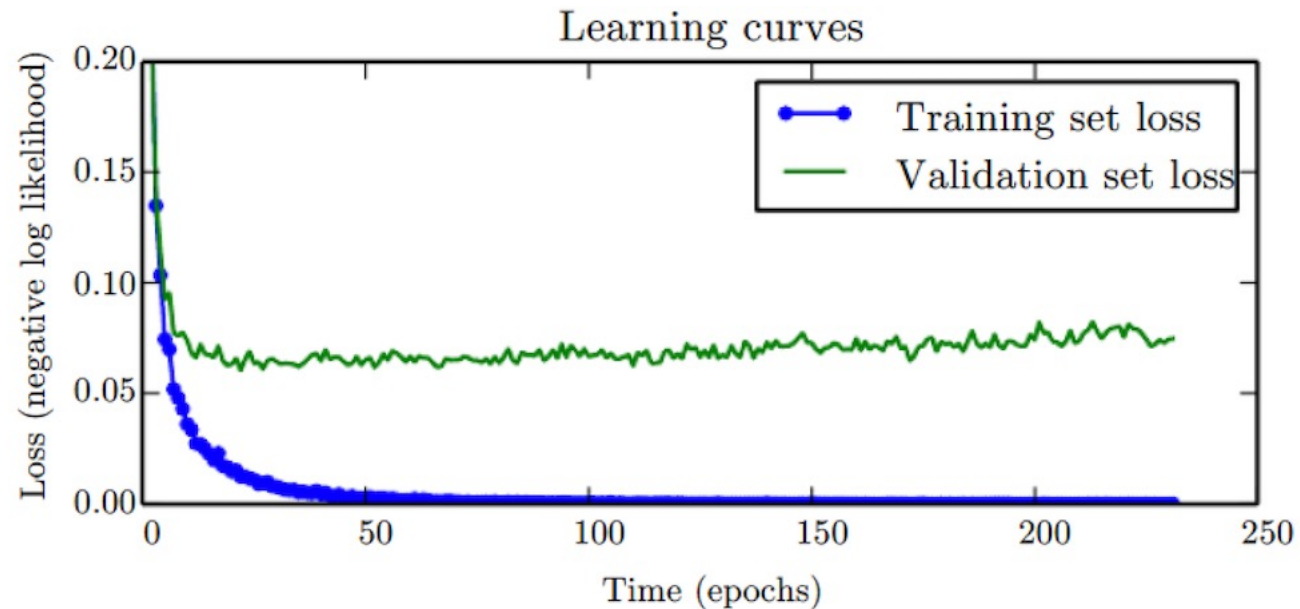


Noise Injection

- Injecting noise in the input can be seen a form of data augmentation
 - Highly effective
 - Can be seen as regularization
- Different ways of noise injection
 - A random noise to input
 - Noise injection to parameters (or models) (~ Tikhonov regularization)
 - Noise at the output targets (e.g. label smoothing)

Early Stopping

- A conventional learning curve (negative log-likelihood)
 - The training objective decreases consistently over time
 - The validation loss forms an asymmetric U-shaped curve
 - e.g., Maxout network on MNIST



Early Stopping

- Idea: use the parameters at the minimum validation error
 - Maintain a copy of model parameters every iteration
 - Additional cost is negligible (+ occasional slow writes)
 - A kind of hyperparameter selection algorithm (training time)
- A very unobtrusive form of regularization
 - No change in training procedure, objective, etc.
 - Can be used jointly with other regularization
 - Regularization effect (decreasing generalization error) + reduction of training time

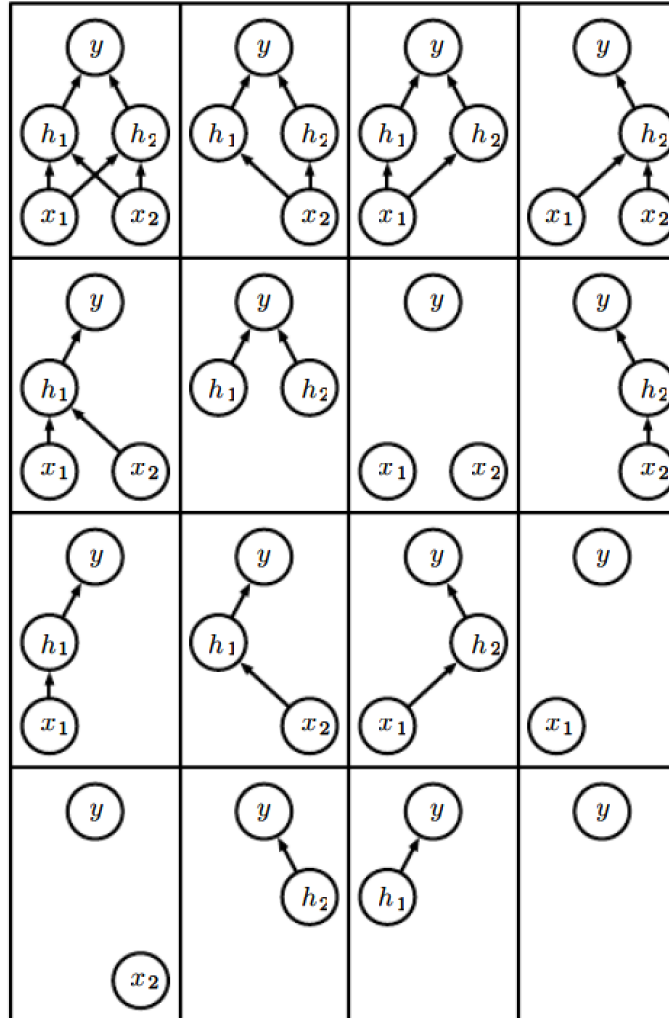
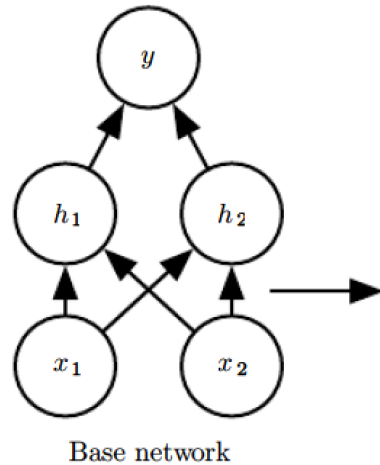
Dropout

- A method of bagging for ensembles of very many large NNs
 - An inexpensive approximation to training and evaluating a bagged ensemble of exponentially many NNs
- Suppose that we have a big NN model and optimize it with a minibatch-based learning algorithm (e.g., SGD)
 - For each minibatch, randomly sample a different binary mask to apply to all of the input and hidden units in the network
 - In other words, randomly drop the output of units to zero
 - e.g., dropout rate 0.5 for hidden units and 0.2 for input nodes
 - Then use learning algorithm as usual

Dropout

- A base network with two input and two hidden units

16 possible subsets



Ignore the ones with no input or no paths to output

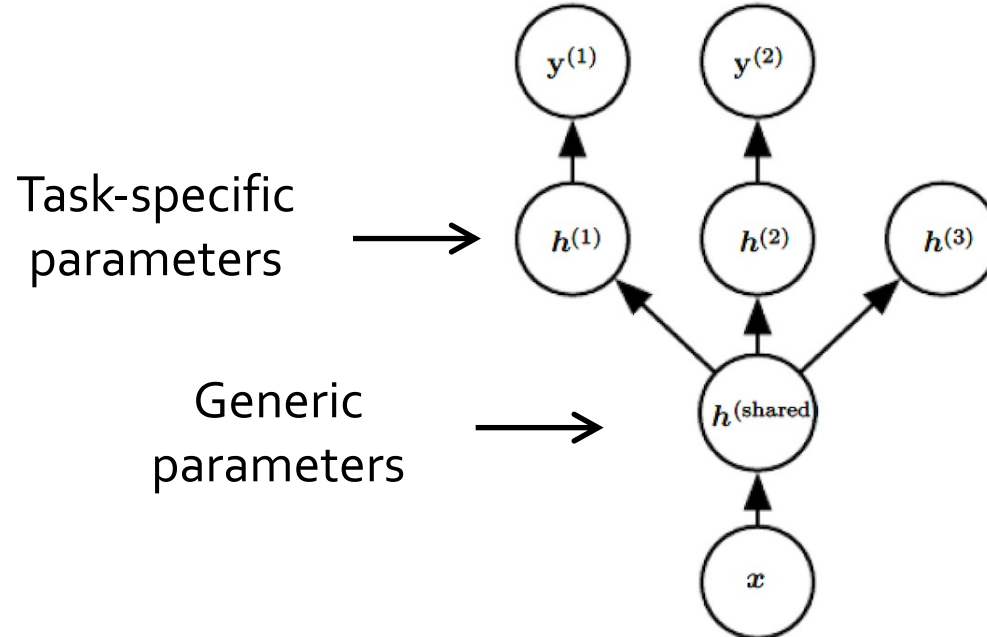
Ensemble of Sub-Networks

Properties of Dropout

- Comparison with Bagging
 - In dropout, the models share parameters
 - With each minibatch, a different subset of parameter is updated
 - In bagging, the models are all independent
- Dropout is very effective and highly recommended
 - More effective than other regularizers (e.g., weight decay, sparsity)
 - Computationally very cheap
 - No limitation on model types or training procedures (e.g., CNNs, RNNs, and RBMs)

Multi-task Learning

- Learn multiple-related problems together at the same time
 - e.g. Image classification and object detection
- Two different supervised tasks (predicting $\mathbf{y}^{(i)}$), while sharing input $\mathbf{x}$ and some mid-level representation $\mathbf{h}^{(\text{shared})}$



Adversarial Training

- Adversarial training: training on adversarially perturbed examples
 - Human cannot tell the difference with the original example
 - However, the network can make highly different predictions

